a file, then that file will be copied from the read-only layer below into the read-write layer. The read-only version of the file will still exist but is now hidden underneath the copy.

This pattern is traditionally called "copy on write" and is one of the features that makes Docker so powerful. Each read-only image layer is read-only; this image never changes. When a container is created, Docker builds from the stack of images and then adds the read-write layer on top. That layer, combined with the knowledge of the image layers below it and some configuration data, form the container. As we discovered in the last chapter, containers can be changed, they have state, and they can be started and stopped. This, and the image-layering framework, allows us to quickly build images and run containers with our applications and services.

Listing Docker images

Let's get started with Docker images by looking at what images are available to us on our Docker host. We can do this using the `docker images` command.

Listing 1.2: Listing Docker images

```
$ sudo docker images
REPOSITORY TAG     IMAGE ID     CREATED     VIRTUAL SIZE
ubuntu     latest  c4ff7513909d 6 days ago  225.4 MB
```

We see that we've got an image, from a repository called `ubuntu`. So where does this image come from? Remember in Chapter 3, when we ran the `docker run` command, that part of the process was downloading an image? In our case, it's the `ubuntu` image.

NOTE Local images live on our local Docker host in the `/var/lib/docker` directory. Each image will be inside a directory named for your storage driver; for example, `aufs` or `devicemapper`. You'll also find all your containers in the `/var/lib/docker/containers` directory.

That image was downloaded from a repository. Images live inside repositories, and repositories live on registries. The default registry is the public registry managed by Docker, Inc., Docker Hub.

TIP The Docker registry code is open source. You can also run your own registry, as we'll see later in this chapter. The Docker Hub product is also available as a commercial "behind the firewall" product called Docker Trusted Registry, formerly Docker Enterprise Hub.

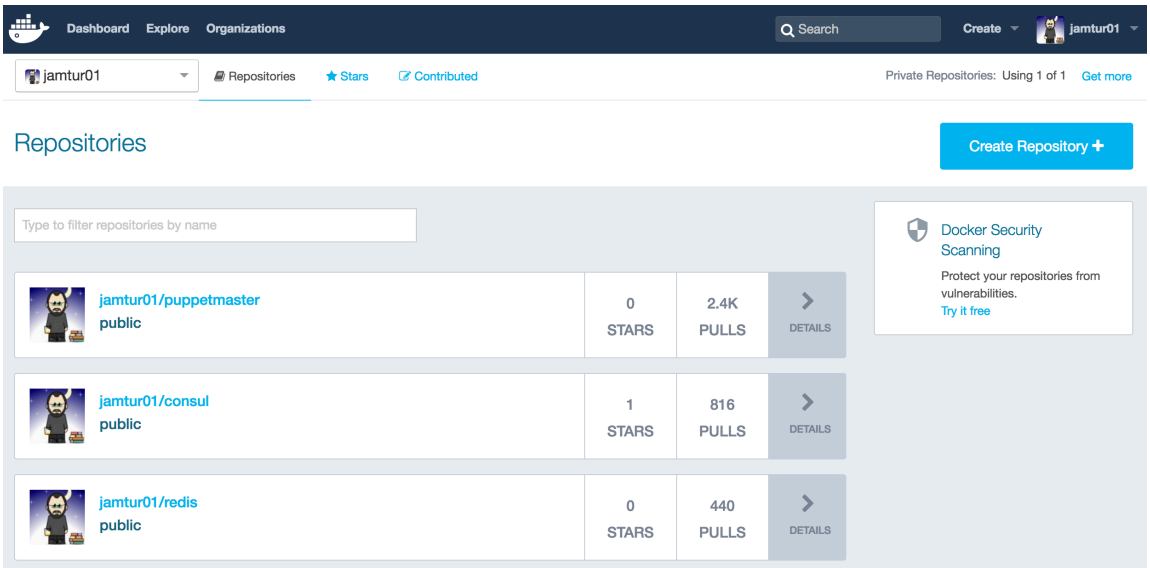

Figure 1.2: Docker Hub

Inside Docker Hub (or on a Docker registry you run yourself), images are stored in repositories. You can think of an image repository as being much like a Git repository. It contains images, layers, and metadata about those images.

Each repository can contain multiple images (e.g., the `ubuntu` repository contains images for Ubuntu 12.04, 12.10, 13.04, 13.10, 14.04, 16.04, 18.04). Let's get another image from the `ubuntu` repository now.

Listing 1.3: Pulling the Ubuntu 18.04 image

```
$ sudo docker pull ubuntu:18.04
18.04: Pulling from library/ubuntu
Digest: sha256:
c6674c44c6439673bf56536c1a15916639c47ea04c3d6296c5df938add67b54b
Status: Downloaded newer image for ubuntu:18.04
```

Here we've used the `docker pull` command to pull down the Ubuntu 18.04 image from the `ubuntu` repository.

Let's see what our `docker images` command reveals now.

Listing 1.4: Listing the ubuntu Docker images

```
$ sudo docker images
REPOSITORY TAG     IMAGE ID     CREATED      VIRTUAL SIZE
ubuntu     latest  5506de2b643b 3 weeks ago  199.3 MB
ubuntu     18.04   0b310e6bf058 5 months ago 127.9 MB
```

TIP Throughout the book we use the `ubuntu` image. This is a reasonably heavyweight image, measuring a couple of hundred megabytes in size. If you'd prefer something smaller the Alpine Linux image is recommended as extremely

lightweight, generally 5Mb in size for the base image. Its image name is `alpine`.

You can see we've now got the `latest` Ubuntu image and the `18.04` image. This shows us that the `ubuntu` image is actually a series of images collected under a single repository.

NOTE We call it the Ubuntu operating system, but really it is not the full operating system. It's a cut-down version with the bare runtime required to run the distribution.

We identify each image inside that repository by what Docker calls tags. Each image is being listed by the tags applied to it, so, for example, `12.04`, `12.10`, `quantal`, or `precise` and so on. Each tag marks together a series of image layers that represent a specific image (e.g., the `18.04` tag collects together all the layers of the Ubuntu 18.04 image). This allows us to store more than one image inside a repository.

We can refer to a specific image inside a repository by suffixing the repository name with a colon and a tag name, for example:

Listing 1.5: Running a tagged Docker image

```
$ sudo docker run -t -i --name new_container ubuntu:18.04 /bin/
bash
root@79e36bff89b4:/#
```

This launches a container from the `ubuntu:18.04` image, which is an Ubuntu 18.04 operating system.

It's always a good idea to build a container from specific tags. That way we'll know exactly what the source of our container is. There are differences, for example, between Ubuntu 16.04 and 18.04, so it would be useful to specifically state that we're using `ubuntu:18.04` so we know exactly what we're getting.

There are two types of repositories: user repositories, which contain images contributed by Docker users, and top-level repositories, which are controlled by the people behind Docker.

A user repository takes the form of a username and a repository name; for example, `jamtur01/puppet`.

- Username: `jamtur01`
- Repository name: `puppet`

Alternatively, a top-level repository only has a repository name like `ubuntu`. The top-level repositories are managed by Docker Inc and by selected vendors who provide curated base images that you can build upon (e.g., the Fedora team provides a `fedora` image). The top-level repositories also represent a commitment from vendors and Docker Inc that the images contained in them are well constructed, secure, and up to date.

In Docker 1.8 support was also added for managing the content security of images, essentially signed images. This is currently an optional feature and you can read more about it on the Docker blog.

WARNING User-contributed images are built by members of the Docker community. You should use them at your own risk: they are not validated or verified in any way by Docker Inc.